

JavaScript - Advanced Concepts

Contents

- ✓ Data types in JavaScript
- ✓ Arrays, Array functions
- ✓ Object Oriented Programming – this, static, prototype
- ✓ Data Structures – Set, Map,
- ✓ Cookie
- ✓ Storage – localStorage, sessionStorage
- ✓ Synchronicity
- ✓ Promise
- ✓ Fetch

Datatypes in JavaScript

- ✓ JavaScript has 5 primitive data types that are passed by value –
 - a. **String**: Represents textual data enclosed in quotes.
Example: "Hello World".
 - b. **Number**: Represents both whole integers and decimal floating-point values. Example: 42 or 3.14.
 - c. **Boolean**: Represents a logical entity and can only be true or false.
 - d. **Undefined**: Assigned automatically to variables that have been declared but not given a value. Example: let x;
 - e. **Null**: Represents an intentional absence of any object value or an empty state. Example: let user = null;
- ✓ Note – ‘**any**’ is not a data type in JavaScript.
- ✓ JavaScript is a dynamically typed language. This means variable types are not declared explicitly. A single variable can naturally hold a string, a number, or any other type during execution.

contd..

- ✓ JavaScript has 3 non-primitive data types that are passed by reference
 - a. **Object** – A collection of key-value pairs used to store structured data. `const user = { name: "Alice", age: 25 };`
 - b. **Array** – An ordered, indexed list used to store multiple values of any type. `const colors = ["red", "green", "blue"];`
 - c. **Function** – A reusable block of executable code designed to perform a specific task.

```
function greet() {  
    return "Hello!";  
}
```

- ✓ In non-primitive data types, there are some challenges for copying data. Objects are created at some place in the computer's memory. When a property is declared, it creates a space in memory.
- ✓ JavaScript always passes by value, but in an array or object, the value is a reference to it, so you can 'change' the data.

Arrays in JavaScript

- ✓ Arrays are used for storing ordered collections.

contd..

Function	Meaning
push(element)	Adds one or more elements to the end of an array and returns the new length.
pop()	Removes and returns the last element from the array.
unshift(element)	Adds one or more elements to the beginning of an array.
shift()	Removes and returns the first element of an array.
includes(value)	Checks whether the specified value exists in the array and returns true or false.
indexOf(value)	Returns the index of the first occurrence of a value; returns -1 if not found.

contd..

<code>lastIndexOf(value)</code>	Returns the index of the last occurrence of a value in the array.
<code>slice(start, end)</code>	Extracts a portion of an array and returns a new array without modifying the original array.
<code>splice(start, count, items)</code>	Adds, removes, or replaces elements in the original array at a specified position.
<code>concat(array2)</code>	Combines two or more arrays and returns a new array.
<code>join(separator)</code>	Converts all array elements into a single string separated by the specified delimiter.
<code>reverse()</code>	Reverses the order of elements in the array.
<code>sort()</code>	Sorts array elements in ascending order (alphabetically for strings by default).
<code>forEach(callback)</code>	Executes a function once for each array element; mainly used for processing/displaying data.

contd..

map(callback)	Creates and returns a new array by applying a function to each element.
filter(callback)	Returns a new array containing only elements that satisfy a given condition.
find(callback)	Returns the first element that satisfies the specified condition.
findIndex(callback)	Returns the index of the first element that satisfies the condition.
some(callback)	Returns true if at least one element satisfies the condition.
every(callback)	Returns true only if all elements satisfy the condition.
reduce(callback, initialValue)	Processes all elements and reduces them to a single value such as a sum, count, or average.
toString()	Converts the entire array into a comma-separated string.

OOP in JavaScript

- ✓ **JavaScript** supports **Object-Oriented Programming** (OOP), but it is based on **prototypes** rather than traditional **classes**. Since **ES6**, JavaScript also provides **class syntax**.
- ✓ OOP is a programming style where **data** and **behavior** are organized into **objects**.
- ✓ An object contains:
 - **Properties (data)** → characteristics of the object
 - **Methods (functions)** → actions the object can perform
- ✓ E.g.

```
let student = { name: "Akash" age: 25,  
                displayInfo: function() {  
                    console.log("Name:", this.name);  
                    console.log("Age:", this.age); } };  
  
student.displayInfo();
```

Prototype concept

- ✓ Originally, JavaScript had **no classes**.
 - ✓ Instead, objects could be created directly and inherit from other objects through a mechanism called **prototypes**.
 - ❑

```
let person = {  
  greet() {  
    console.log("Hello");  
  }  
};
```
 - ❑ person object contains greet().
 - ❑ student object is created using Object.create(person).
 - ❑ student does not have its own greet() method.
 - ❑ JavaScript looks in its prototype (person) and finds greet().
- ```
let student = Object.create(person);
student.name = "Vijay";
student.greet();
```

## contd..

- ✓ **Objects** in **JavaScript** may be defined as an **unordered collection** of **related data**, of **primitive** or **reference types**, in the form of “**key: value**” pairs. These **keys** can be **variables** or **functions** and are called **properties** and **methods**, respectively.
- ✓ **Object** is a reference **data type**. Variables that are assigned a reference value are given a reference or a pointer to that value. That reference or pointer points to the location in memory where the object is stored. The variables don't actually store the value.
- ✓ **Objects** are more complex and each object contains any combination of these **primitive data-types** as well as **reference data-types**.
- ✓ **Objects** are used for storing keyed collections.

# this

- ✓ In JavaScript, **this** is a special keyword that refers to the object that is currently executing the function.
- ✓ this is a reference to the object that invokes the function. Its value is determined at runtime based on how the function is called.
- ✓ Unlike languages such as Java or C#, the value of this in JavaScript is determined by **how a function is called**, not where it is defined.
- ✓ **Note** – Arrow functions **do not create their own this**. They inherit this from the surrounding scope.
- ✓ For object methods, avoid arrow functions unless you specifically need lexical this.

# static

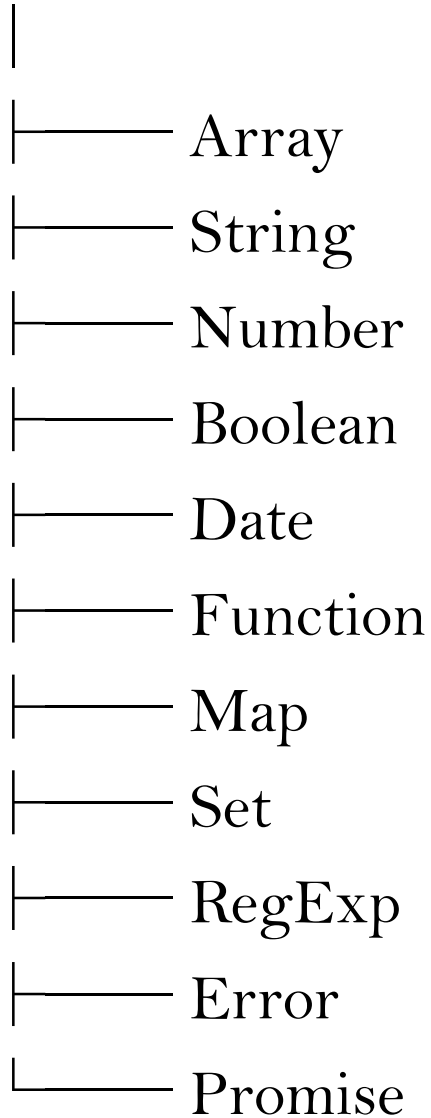
- ✓ In JavaScript OOP, **static** means a property or method belongs to the **class** itself, **not to objects (instances)** created from that class.
- ✓ **static properties** are called by the **class** and not by individual instances.

## contd..

```
let school =
{
 name: 'Vivekananda School',
 location : 'Delhi',
 established : '1971',
 displayInfo : function()
 {
 document.write(`${school.name} was
 established in ${school.established} at
 ${school.location}`)
 }
}
school.displayInfo();
```

# Object prototype

Object



# Set

- ✓ **Set** is a collection of **unique values**.
  - ❑ Duplicate values are not allowed.
  - ❑ Values can be of any type (number, string, object, etc.).
  - ❑ Elements are stored in insertion order.
- ✓ **Creating a Set**
  - ❑ `let numbers = new Set();`
  - ❑ Or
  - ❑ `let numbers = new Set([10, 20, 30, 40]);`
- ✓ **Adding Elements**
  - ❑ `let numbers = new Set();`  
`numbers.add(10);`  
`numbers.add(20);`  
`numbers.add(30);`  
`numbers.add(20); // Duplicate ignored`  
`console.log(numbers);` Output: `Set(3) {10, 20, 30}`



## ✓ Important Methods

### ❑ **add()**

```
numbers.add(40);
```

### ❑ **delete()**

```
numbers.delete(20);
```

### ❑ **has()**

Checks whether an element exists.

```
console.log(numbers.has(10)); Output: true
```

### ❑ **size**

Returns number of elements.

```
console.log(numbers.size); Output: 3
```

### ❑ **clear()**

Removes all elements.

```
numbers.clear();
```

## ❑ Traversing a Set

Using for...of

```
let colors = new Set(["Red", "Green", "Blue"]);
```

```
for(let color of colors)
{
 console.log(color);
}
```

Output: Red Green Blue

## ❑ Example: Remove Duplicate from Array

```
let arr = [10,20,10,30,20,40];
let uniqueValues = [...new Set(arr)];
console.log(uniqueValues);
```

Output:

```
[10,20,30,40]
```

## contd..

- ✓ **WeakSet** in JavaScript is used to store a collection of objects.
- ✓ It adapts the same properties of that of a set i.e. does not store duplicates.
- ✓ The major difference of a WeakSet with the set is that a WeakSet is a collection of objects and not values of some particular type.
- ✓ **Syntax:**  
new WeakSet(object)

# Map

- ✓ **Map** is a collection of elements where each element is stored as a *Key-value* pair.
- ✓ **Map** object can hold both *objects* and *primitive* values as either key or value.
- ✓ When developer iterate over the map object it returns **the key, value pair** in the same order as inserted.
- ✓ **new Map([it])**
- ✓ **Map.prototype.size** – It returns the number of elements or the key-value pairs in the map.
- ✓ **Map.prototype.set()** – It adds the key and value to the Map Object.

## contd..

- ✓ WeakMap object in JavaScript is used to store the collection of objects (key-value pairs) in which keys are weakly referenced.
- ✓ The major difference of a WeakSet with the set is that a WeakSet is a collection of objects and not values of some particular type.

## contd..

- ❑ set()
- ❑ get()
- ❑ has()
- ❑ delete()
- ❑ clear()
- ❑ size
- ❑ keys()
- ❑ values()
- ❑ entries()
- ❑ forEach()

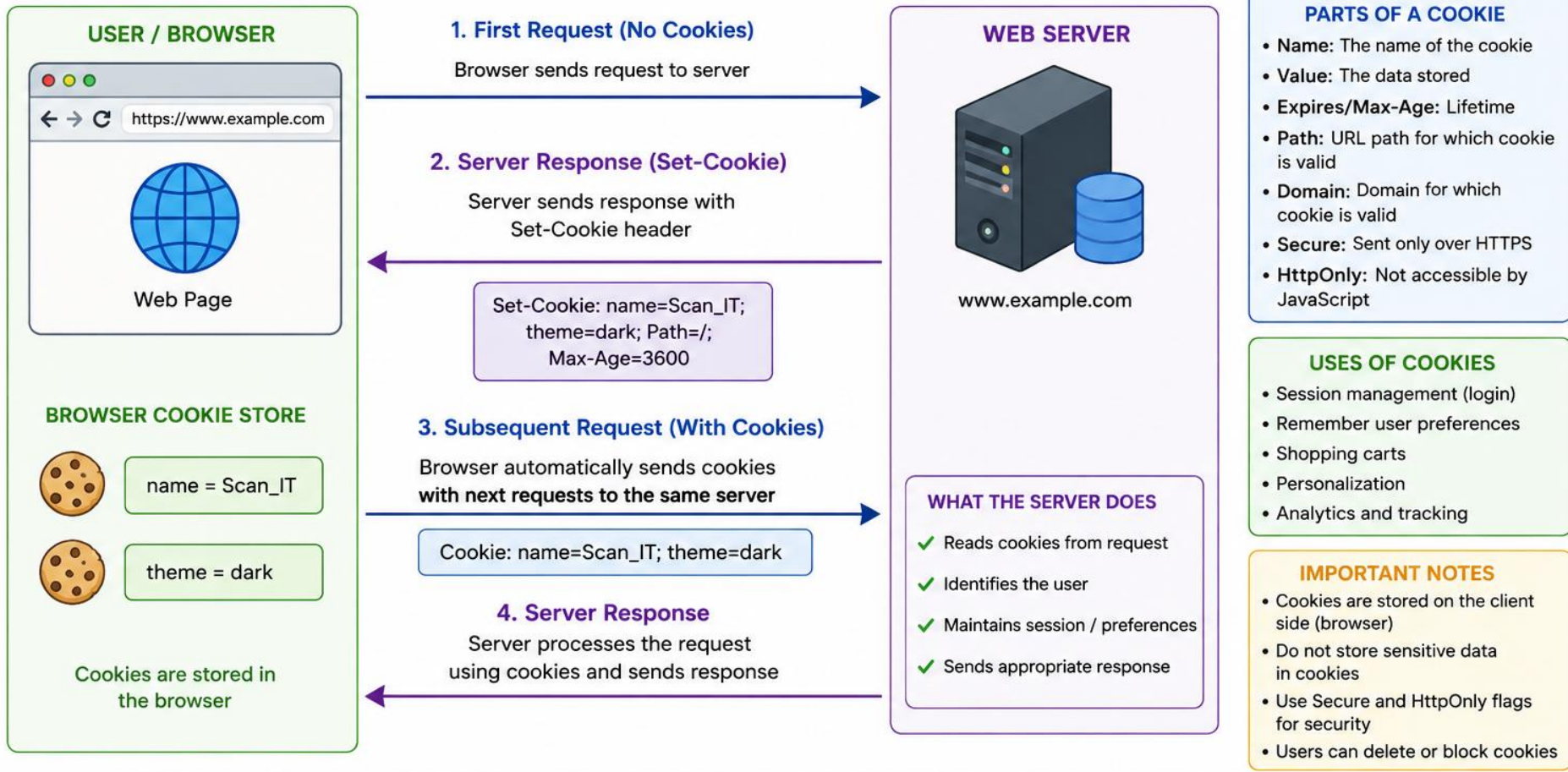
# Summary of Data Structures in JS

| Sl.No. | Data Structure | Built-in? | Typical Implementation   |
|--------|----------------|-----------|--------------------------|
| 1      | Array          | Yes       | `[]`                     |
| 2      | Object         | Yes       | `{}`                     |
| 3      | Set            | Yes       | `new Set()`              |
| 4      | Map            | Yes       | `new Map()`              |
| 5      | Stack          | No        | Array + `push()/pop()`   |
| 6      | Queue          | No        | Array + `push()/shift()` |
| 7      | Linked List    | No        | Custom Class             |
| 8      | Tree           | No        | Custom Class             |
| 9      | Graph          | No        | Custom Class             |

# Cookie

## COOKIES IN WEB DEVELOPMENT

Cookies are small text files stored in the user's browser by a website. They are used to remember information about the user across multiple requests and pages.



### KEY POINTS



Cookies help websites remember information about users. They are simple, lightweight and widely used, but should be used carefully for security and privacy.



**Example:** After login, a cookie can be used to keep you logged in so that you don't have to enter your credentials on every page.



## contd..

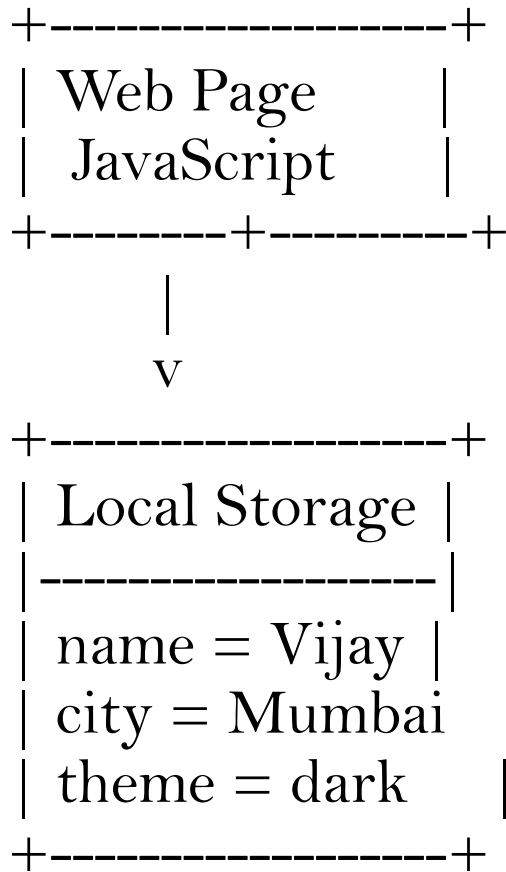
- ✓ JavaScript does not provide a dedicated **Cookie class** with methods like Array or String. Cookies are managed through the `document.cookie` property.
- ✓ The common cookie operations are:
  - ❑ Create a cookie
  - ❑ Read all cookies
  - ❑ Read a specific cookie
  - ❑ Update a cookie
  - ❑ Delete a cookie

# storage in JavaScript

- ✓ **localStorage** is a browser feature, built-in browser object that allows JavaScript to store data as **key-value pairs** in the user's browser. Think of it as a small database inside the browser. The data –
  - ❑ Remains stored even after the browser is closed.
  - ❑ Remains available after the computer is restarted.
  - ❑ Is specific to a website/domain.
  - ❑ Can only store strings (objects must be converted to JSON).
- ✓ The commonly used methods are:
  - ❑ `setItem()` – Add/Store data
  - ❑ `getItem()` – Retrieve data
  - ❑ `removeItem()` – Remove a specific item
  - ❑ `clear()` – Remove all items
  - ❑ `key()` – Get the key at a given index
  - ❑ `length` – Number of stored items

- ✓ **Why Use Local Storage?**
- ✓ Suppose a user enters:
- ✓ Username
- ✓ Theme preference (dark/light mode)
- ✓ Shopping cart items
- ✓ Application settings
- ✓ You can save this information locally and retrieve it later without a server request.

# Visual Representation



Close Browser



Open Browser Again



Data Still Exists

## contd..

1. Reading Data - `getItem()`
2. Updating Data - `localStorage.setItem("name", "Guru");`
3. Removing One Item - `removeItem()`  
`localStorage.removeItem("name");`
4. Removing All Data - `clear()`
5. `localStorage.clear()` - All local storage entries for that website are removed.

contd..

| Method             | Purpose                |
|--------------------|------------------------|
| setItem(key,value) | Store data             |
| getItem(key)       | Read data              |
| removeItem(key)    | Delete one item        |
| clear()            | Delete all items       |
| key(index)         | Get key name by index  |
| length             | Number of stored items |

# contd..

Roundcube Webmail :: Welcome

mail.vinsys.com/mail/?\_task=login&\_err=session

Username

Password

LOGIN

Roundcube Webmail

! Your session is invalid or expired.

Application

- Manifest
- Service workers
- Storage

Storage

- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies
- Private state tokens
- Interest groups
- Shared storage
- Cache storage
- Storage buckets

Background services

- Back/forward cache
- Background fetch
- Background sync
- Bounce tracking miti...
- Notifications
- Payment handler
- Periodic background ...

Usage

0 B used out of 173,841 MB storage quota

[Learn more](#)

0 B ☐ Total

☐ Simulate custom storage quota

Application

- ☒ Unregister service workers

Storage

- ☒ Local and session storage
- ☒ IndexedDB
- ☒ Cookies
- ☒ Cache storage

contd..

| Feature        | Local Storage | Cookies                  |
|----------------|---------------|--------------------------|
| Capacity       | ~5-10 MB      | ~4 KB                    |
| Sent to Server | No            | Yes (with HTTP requests) |
| Expiry         | Until removed | Can expire               |
| Storage Type   | Key-Value     | Key-Value                |
| Performance    | Faster        | Slower                   |
|                |               |                          |

## Check Local Storage in Chrome

- 1.Open web page.
- 2.Press **F12**.
- 3.Open **Application** tab.
- 4.Expand **Local Storage**.
- 5.Select your website.



contd..

| Feature                | Local Storage     | Session Storage      |
|------------------------|-------------------|----------------------|
| Survives Browser Close | Yes               | No                   |
| Survives Restart       | Yes               | No                   |
| Shared Across Tabs     | Yes (same origin) | No (per tab/session) |
| Capacity               | ~5-10 MB          | ~5 MB                |
| Feature                | Local Storage     | Session Storage      |
| Survives Browser Close | Yes               | No                   |

## contd..

- ✓ localStorage stores data in the browser as **key-value pairs**.
- ✓ Data persists even after closing the browser.
- ✓ Main methods:
  - ✓ setItem()
  - ✓ getItem()
  - ✓ removeItem()
  - ✓ clear()
- ✓ Stores only strings; use JSON.stringify() and JSON.parse() for objects.
- ✓ Commonly used for user preferences, settings, and client-side application data.

# sessionStorage

- ✓ sessionStorage is a browser storage mechanism that stores data as **key-value pairs**,

| Feature                  | localStorage            | sessionStorage                              |
|--------------------------|-------------------------|---------------------------------------------|
| Lifetime                 | Permanent until removed | Exists only for current browser tab/session |
| Shared Across Tabs       | Yes (same origin)       | No                                          |
| Storage Size             | ~5-10 MB                | ~5 MB                                       |
| Survives Browser Restart | Yes                     | No                                          |

## ✓ **Methods of sessionStorage**

- ❑ `setItem()`
- ❑ `getItem()`
- ❑ `removeItem()`
- ❑ `clear()`
- ❑ `key()`
- ❑ `length` property

# Variable Shadowing

- ✓ Variable shadowing in JavaScript occurs when a variable **declared** in an **inner scope** has the same name as a variable declared in an outer scope.
- ✓ The inner variable effectively "**shadows**" or **hides** the outer variable within the inner scope, meaning that any reference to that variable name within the inner
- ✓ **Scope** will resolve to the inner variable.

# Synchronicity

- ✓ **Synchronous (or synchronicity) execution** means JavaScript executes statements **one after another in sequence**. The next statement waits until the current statement finishes.
  - ❑ Think of it like standing in a queue at a ticket counter:
  - ❑ Person 1 finishes → Person 2 is served
  - ❑ Person 2 finishes → Person 3 is served
  - ❑ Nobody can skip ahead.
- ✓ **Key Points**
  - ❑ JavaScript executes code line by line by default.
  - ❑ One statement finishes before the next starts.
  - ❑ Functions are executed completely before control returns to the next statement.
  - ❑ Long-running tasks can make the program wait.
  - ❑ **Synchronous = "Do one task completely, then start the next task."**

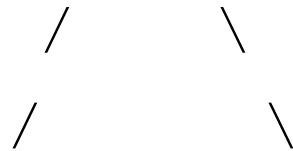
# Promise

- ✓ A **Promise** in JavaScript is an object that represents the **eventual result of an asynchronous operation**.
- ✓ Think of it as a "promise" that:
  - ❑ A task is started now.
  - ❑ The result may come later.
  - ❑ When it finishes, it either:
    - ❑ **Succeeds** → returns a value.
    - ❑ **Fails** → returns an error.
- ✓ Use Case – Consider downloading data from a server.
  - ❑ `let data = getDataFromServer();`  
`console.log(data);`
  - ❑ The download may take a few seconds. JavaScript cannot wait indefinitely because it is single-threaded.
  - ❑ Instead, the operation runs asynchronously and returns a Promise immediately.

# contd..

## ✓ Promise States –

### ❑ Pending



Fulfilled      Rejected

- ❑ Pending: Initial state Operation is still running.
- ❑ Fulfilled: Operation completed successfully
- ❑ Rejected: Operation failed
- ❑ Chaining



# Creating a Promise

✓ `new Promise((resolve, reject) =>`

`{`

`if(success)`

`resolve('Done');`

`else`

`reject('Error');`

`});`

✓ `resolve()` – Marks the Promise as fulfilled and passes a value to `then()`.

✓ `rejectt()` – Marks the Promise as rejected and passes an error to `catch()`.

## contd..

- ✓ **then()** – Executes when the Promise is fulfilled. Can be chained for multiple operations.
- ✓ **catch()** – Handles errors from rejected Promises or exceptions in `then()`.
- ✓ **finally()** – Runs regardless of success or failure. Used for cleanup activities.
- ✓ **Promise** = Async result
- ✓ **States:** Pending, Fulfilled, Rejected
- ✓ **Methods:** `then()`, `catch()`, `finally()`

# Promise Example

```
let p = new Promise((resolve, reject)=>
{
 resolve('Success');
});
p.then(msg => console.log(msg));
```

# Fetch API

- ✓ `fetch()` is used to make HTTP requests and returns a Promise.
- ✓ E.g.
  - ❑ `fetch('https://api.example.com/data')`
    - `.then(response => response.json())`
    - `.then(data => console.log(data))`
    - `.catch(error => console.log(error));`
- ✓ Promise Chaining - `then()` returns a Promise, allowing multiple asynchronous steps to be linked.
- ✓ Fetch API returns a Promise

Thank You